Instituto Tecnológico y de Estudios Superiores de Monterrey

TC2037 - Implementación de Métodos Computacionales

Ian Julián Estrada Castro – A01352823

Actividad Integradora 1 – Convertir una expresión regular a DFA

15 de marzo de 2024

Tabla de contenido

Expresiones *3*
Expresiones regulares *3*
Autómata *3*
Autómata Finito No Determinista (AFN) **4**
Autómata Finito Determinista (AFD) **4**
Actividad Integradora *5*
Expresión regular en formato postfijo *5*
Algoritmo de Thomson *5*
Transformar AFN a AFD *9*
Cerradura épsilon y Move **9**
Reflexión *11*

Expresiones

Secuencia de operandos unidos por operadores para dar un resultado. En una expresión los operadores pueden ser unarios o binarios los cuales tienen una prioridad para su ejecución.

Expresiones regulares

Notación de expresiones que permite describir patrones válidos en secuencias simbólicas. Siendo una expresión, las expresiones regulares cuentan con operandos y operadores básicos:

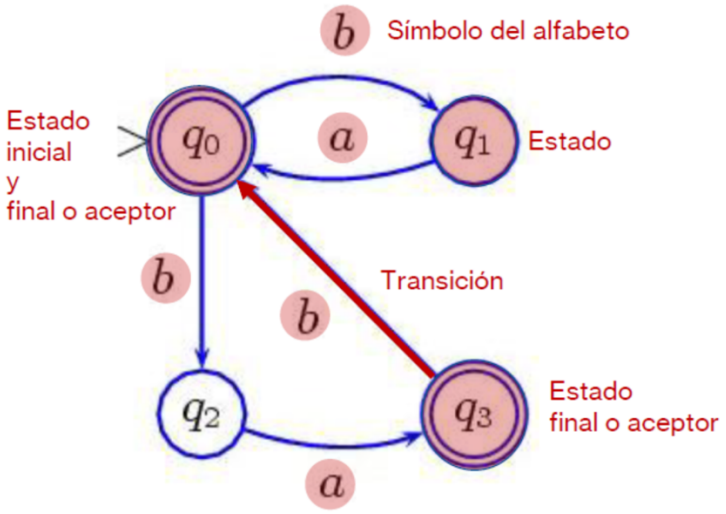
- Operandos
  - Símbolos del alfabeto
  - ε (representan una cadena vacía)
- Operadores
  - Concatenación
  - Alternativa (|)
  - Repetición
    - Cerradura de Kleen (*)
    - Cerradura positiva (+)
  - Paréntesis "()"

Autómata

Modelo matemático funcional que sirve para describir un proceso de transformación (computacional) de una entrada en una salida. Se categorizan dependiendo del tipo de problemas que pueden resolver (capacidad o potencia computacional):

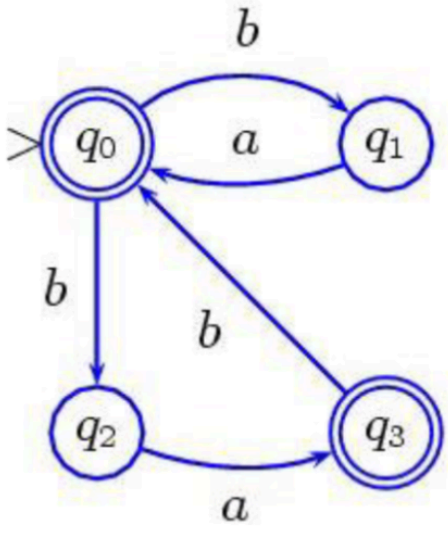
- Autómatas Finitos
- Autómatas de pila
- Máquinas de Turing

Un autómata finito puede describir con base en los estados del proceso y las transiciones entre los estados según los datos de entrada, hasta llegar a un estado de aceptación o rechazo. Los autómatas finitos pueden ser finitos deterministas (AFD) o no deterministas (AFN).



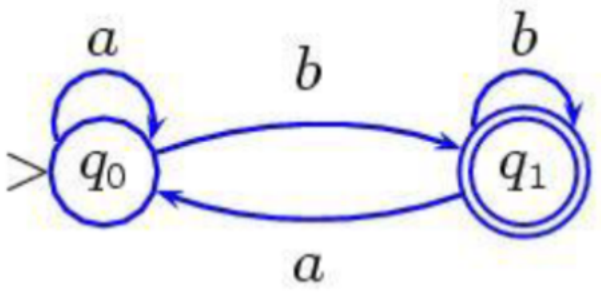
Autómata Finito No Determinista (AFN)

En un autómata No Determinístico pueden existir varias transiciones para un mismo estado y símbolo de entrada. Pueden existir transiciones épsilon y son más fáciles de diseñar.



Autómata Finito Determinista (AFD)

En un autómata Determinista, para cada estado y símbolos de entrada, existe una transición única a otro estado. No hay ambigüedades para implementar.



Actividad Integradora

La actividad consistió en desarrollar un programa que permitiera convertir una expresión regular a un Autómata Finito Determinista. El programa debió implementar el algoritmo visto en la unidad de formación:
Expresión regular → Autómata Finito No Determinista → Autómata Finito Determinista

El programa desarrollado recibe una expresión regular y la tranforma a un AFD a través del algoritmo visto en la unidad de formación. Antes de iniciar el algorimo, es necesario convertir las concatenaciones a un carater explícito y específico, para este caso se utilizó el símbolo de grado (°) para representar las concatenaciones. Esto hace que en medio de las concatenaciones se escriba el símbolo '°' que ahora será utilizado como el operador concatenación.

Expresión regular en formato postfijo

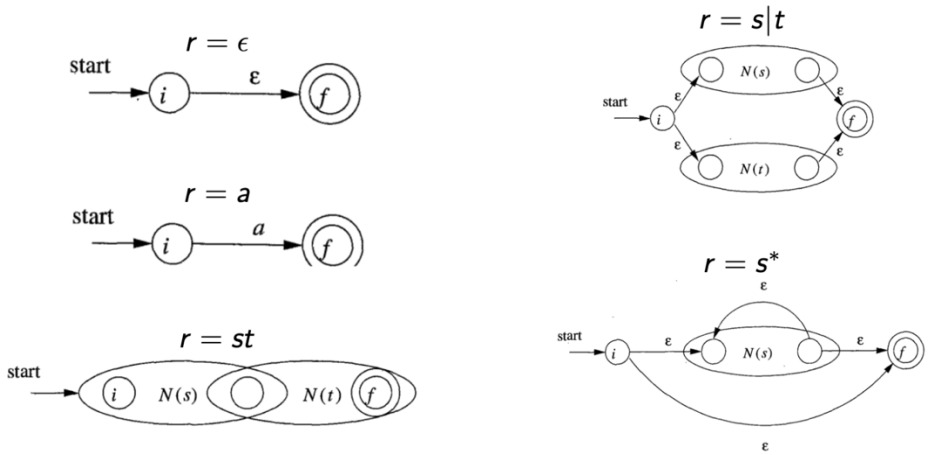
En la notación postfija se representan las operaciones de forma que primero se escriben los operandos y después las operaciones, por ejemplo:

- Notación Infija: a | b
- Notación postfija: ab|

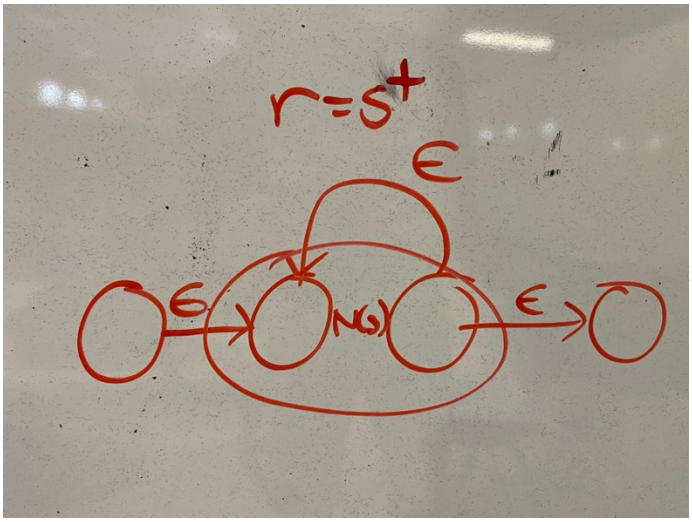
Esta notación será útil ya que primero se construiran las transiciones bajo cierto símbolo y después las operaciones. El primer paso es convertir la expresión regular a un AFN, para ello se utiliza el algoritmo de Thompson.

Algoritmo de Thomson

Desarrollado por Ken Thompson y Dennis Ritchie, el algoritmo de Thomson sirve para construir autómatas finitos no deterministas con transiciones épsilon a partir de expresiones regulares. Posteriormente se utiliza el AFN para construir el AFD. Los operadores de la expresión regular tienen una conversión a AFN a través de este algoritmo:



La cerradura positiva es muy similar a la cerradura de Kleene con excepción de una transición:

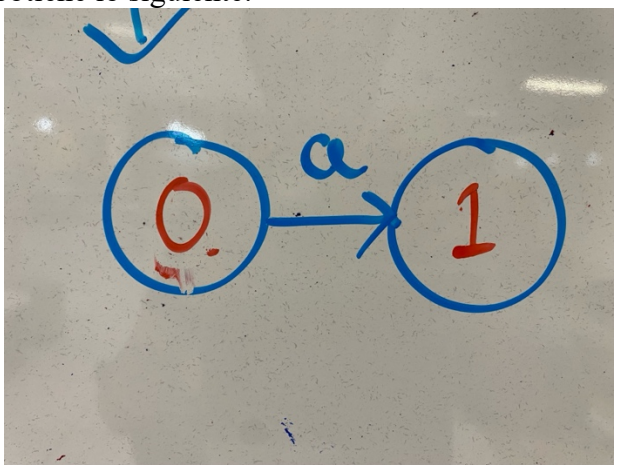


Gracias a la notación postfija, las primeras transiciones que se crean en el AFN son las transiciones bajo los símbolos del alfabeto en la expresión regular. Utilizaremos un ejemplo para crear un autómata:

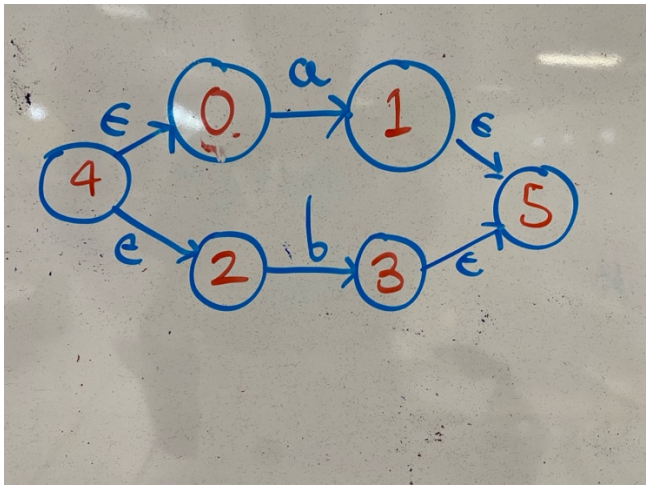
- Expresión regular: (a|b)*abb
- Expresión con concatenaciones convertidas: (a|b)*°a°b°b
- Expresión en notación postfija: ab|*a°b°b°

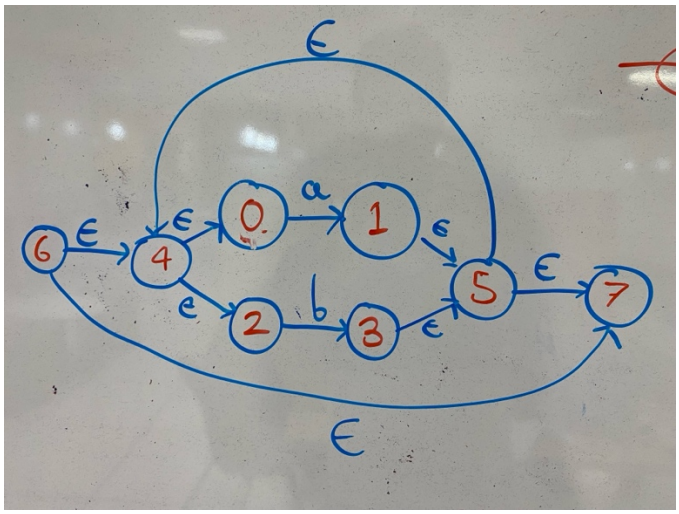
```
RegEx: (a|b)*abb
RegEx_modificado: (a|b)*°a°b°b
RegEx en formato posfijo: ab|*a°b°b°
```

A partir de la expresión regular en notación postfija, se contruye el automata. Por cada símbolo, se genera una o varias transiciones dependiendo del caso. En el primer caso es un símbolo, entonces se obtiene lo siguiente:

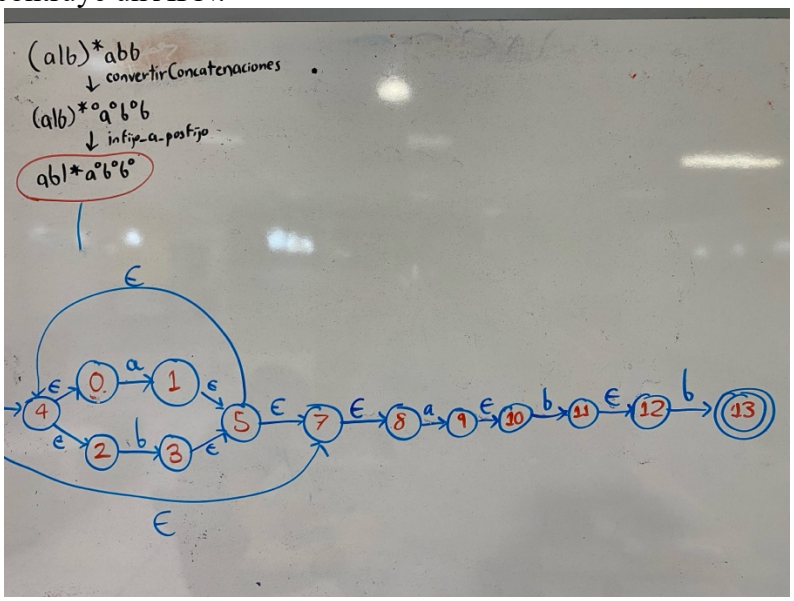


Ocurre el mismo caso con b. Posteriormente se construyen las transiciones dependiendo del operador siguiendo el algoritmo de Thompson, los siguientes casos con alternativa y cerradura de Kleene:





Tras seguir las conversiones de expresión regular a autómata con base en el algoritmo de Thompson, se contruye un AFN:



```
Estado inicial:  6
Estado final:  13
NFA
Transición:  0  ->  [(1, 'a')]
Transición:  1  ->  [(5, '#')]
Transición:  2  ->  [(3, 'b')]
Transición:  3  ->  [(5, '#')]
Transición:  4  ->  [(2, '#'), (0, '#')]
Transición:  5  ->  [(7, '#'), (4, '#')]
Transición:  6  ->  [(4, '#'), (7, '#')]
Transición:  7  ->  [(8, '#')]
Transición:  8  ->  [(9, 'a')]
Transición:  9  ->  [(10, '#')]
Transición:  10  ->  [(11, 'b')]
Transición:  11  ->  [(12, '#')]
Transición:  12  ->  [(13, 'b')]
Accepting State:  13
```

Transformar AFN a AFD

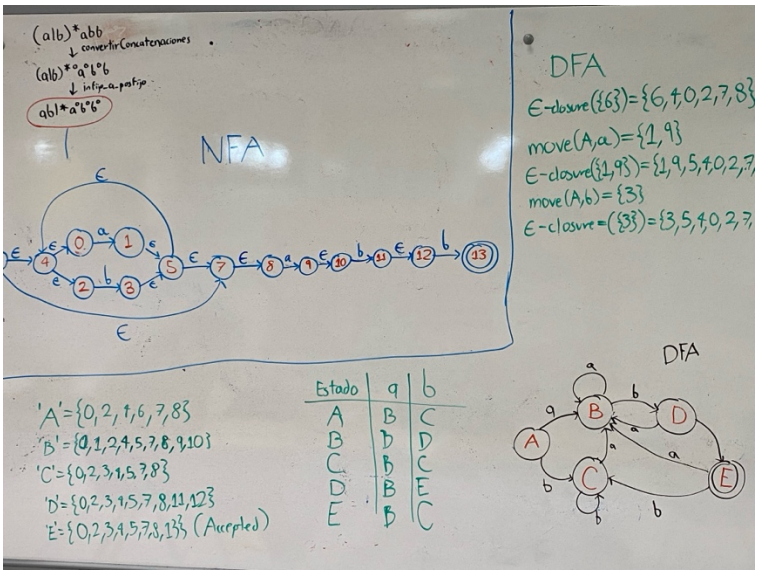
Para construir un Autómata Finito Determinista (AFD), es necesario haber construido previamente el AFN. Una vez hecho, el AFD se construye a través de dos funciones importantes: *Cerradura épsilon* y *move*.

Cerradura épsilon y Move

Cerradura épsilon obtiene todos los estados alcanzables bajo épsilon creando un conjunto de estados. Cerradura épsilon debe aplicarse al estado inicial, a partir de ahí a los conjuntos que aparezcan se les aplica la función move. Move se aplica a cada conjunto bajo cierto símbolo, cuando se aplique, pueden ocurrir tres casos:

- El conjunto es nuevo
- El conjunto ya existe
- El conjunto está vacío

Sólo en el primer caso se le asigna un nuevo valor que llamamos *etiqueta*, si un conjunto ya existe, se le asigna la misma etiqueta, si el conjunto está vacío, no se toma en cuenta. Ambos procesos se aplican hasta que ya no aparezcan nuevos conjuntos. Una vez obtenidos los conjuntos y su etiqueta, se construye una tabla de estado donde se escriben los estados y los estados que van bajo cierto símbolo. La etiqueta cuyo conjunto contenga el estado final (estado de aceptación), será un estado de aceptación. Con base en la tabla de estados, se contruye el AFD:




```
A  ->  {0, 2, 4, 6, 7, 8}
B  ->  {0, 1, 2, 4, 5, 7, 8, 9, 10}
C  ->  {0, 2, 3, 4, 5, 7, 8}
D  ->  {0, 2, 3, 4, 5, 7, 8, 11, 12}
E  ->  {0, 2, 3, 4, 5, 7, 8, 13}
DFA
A  -> [( B :  a )( C :  b )]
B  -> [( B :  a )( D :  b )]
C  -> [( B :  a )( C :  b )]
D  -> [( B :  a )( E :  b )]
E  -> [( B :  a )( C :  b )]
Accepting state/s:  ['E']
```

Reflexión

Aunque al principio la programación de un AFN sonó complicada, en realidad fue más sencillo al entender la lógica del algoritmo de Thompson. Desarrollarlo desde su contrucción en lápiz y papel hasta la cadifición en el lenguaje de programación Pyhton ayudó a desarrollar la lógica y entendimiento de un autómata, tanto AFN como AFD, además, fue de gran apoyo para reforzar el manejo del lenguaje y conceptos previos tales como grafos y recorridos. También aprendí nuevas funciones propias de Python como las listas set(), reforcé el uso de estructuras como pilas y aprendí nuevos operadores como la unión. Aun con ello, el programa realizadó resultó con una complejidad promedio de O(n^2), lo que lo vuelve menos eficiente frente a otros algoritmos. Esto deja un área de oportunidad para mejorarlo y reducir la complejidad promedio, y para ello tendré que indagar más en el lenguaje, así como aprender y buscar métodos más eficientes a lo que yo he aplicado.